Project Overview

It's time to design and grow a Java application using typical industry practices!

Learning Objectives

- Design and create code following a layered architecture
- Apply design patterns where appropriate
- Experience the team software development process using industry-standard tools and approaches

Choosing a Project

You and your team will choose a project from a list of project domains (you can invent your own), create user stories as a team, draw the initial user interface as a team, and then start to grow your project. Everyone will contribute code — you'll each be responsible for implementing 1 or more user stories. Here is a **list of project ideas (https://q.utoronto.ca/courses/444184/pages/project-domains-and-sample-user-stories)** .

Step by Step

To give you an idea of what you'll be experiencing this term, here are the steps that you will be taking to complete your project.

- Choose a project domain (one of several examples or your own!)
- Identify one or more existing APIs that can be leveraged for some interesting aspects of your program
- Create a set of User Stories that describe how the application will be used
- Repeatedly:
  - choose a User Story
  - figure out how to modify the UI to incorporate it (are there new buttons and text fields and so on? where?)
  - create a Use Case and figure out the exact user interactions to allow the user story to happen
  - figure out what new code needs to be written (including unit tests) and write it
  - submit a pull request
- Take part in code reviews to review your team's code
- Continually refactor your code to improve the design and adherence to Clean Architecture and the SOLID design principles — applying what you are learning in lectures
- Identify future extensions to make your software even better

In short, you will experience the software design process!

Don't worry if this all sounds daunting. You'll have a TA and your whole team helping you along the way! **Cross-team discussions, visiting office hours, and being active on Piazza are strongly**

encouraged!

Some Process Notes

The project will be completed before the end of term with weekly guidance to help ensure your group is making progress. Later labs will be oriented toward making progress on the course project. The primary deliverable will be a group presentation of your project.

Don't be afraid to make mistakes or write badly-formatted/styled code, but expect to fix it later! Refactoring is part of the process and **we want to see your team working through the software design process**. Yes, you should have a functional piece of software to show off at the end of the term, but more importantly you should be able to:

- tell us about its design,
- explain ***why*** your team chose to design it that way,
- describe how you might improve it in the future, if given more time, and
- **demonstrate effective software development practices, which you will use beyond this course!**

By the end of the term, your team's repository should be a snapshot of the design process. Ideally, there will not only be well-documented and well-styled code, a design that follows Clean Architecture, design patterns used where appropriate, and **git logs demonstrating significant contributions from the whole team.** There may also be open issues left to be resolved, new features you would like to implement, and notes about design patterns you believe could be incorporated in the future.

We encourage every group to make their project their own by incorporating unique features and being creative. **You can talk about your unique project experiences in future interviews for internships, so we hope that you make the most of the opportunities this project can provide.**

Teamwork and Professionalism

Part of your grade will be determined by how well you functioned as a member of your team. Were you respectful in your interactions with your teammates? Did you act professionally during group meetings? Did you do your fair share of the work? Did you meet deadlines? Did you communicate with your team when you encountered delays? Were you a good team member?

Your TA will be assessing this based on their weekly interactions with your team, peer evaluations, and the contents of your team's git repository.

Please communicate with the course team if you are experiencing challenges or tension within your team that can't be resolved internally. The sooner we hear from you the more we can do to help!

Did you know?

"Several kinds of people skills were ranked extremely high by the participants—usually higher than C++ and other technical skills. Several participants mentioned that assessing culture fit was what they considered the primary goal of interviews, with technical qualifications being secondary. Technical skills could even be learned on the job."

— quote from **What game developers look for in a new graduate: interviews and surveys at one game company (https://dl.acm.org/doi/10.1145/1734263.1734359)**

Many of the most important skills for entry-level software developer roles are not technical at all. Instead, they are the kinds of self-directed learning and teamwork skills you will be developing throughout this course project.

As an example, the research article quoted above found that one of the top qualifications for new hires at a game development company was the ability to collaborate effectively and “check your ego at the door”. In other words, being open to feedback, working well with others, and staying humble are just as critical as writing good code.